

Gate-Time Delay Risk

Streaming flight-delay scoring and alerts, at scheduled gate time. MSDS 682 final project, written report. Aidan Percy and Sebastian Steen, August 14, 2026. Code and review path: <https://github.com/sebaleks/flight-delay-stream>

1. Problem and useful result

Delay information is retrospective: ops teams learn about delays after the fact. The actionable moment is scheduled departure time; afterwards it is too late to re-plan gates, crews, or connections. The problem: score every US domestic departure, at its scheduled gate time, with the probability it will arrive 15 or more minutes late, using only information knowable at that moment.

The target user is airport and airline operations teams deciding gate assignments, crew moves, and passenger handling. The starting asset is the XGBoost arrival-delay classifier built in the predecessor 681 project (held-out ROC-AUC 0.74), frozen at its 2024-06-30 training run and never retrained. The scope is one held-out week of BTS flights (2024-09-02 to 2024-09-08, 151,878 departures), replayed through Kafka in departure order.

The observable result is a delay-risk score for every departure published to a Kafka risk topic, plus an alert artifact for the riskiest flights: `data/reference_output/alerts.jsonl`, 7,061 alerts at the 0.5 threshold. A representative alert, the week's highest-risk flight:

```
{"carrier": "WN", "flight_number": "3460", "flight_date": "2024-09-08",  
  "origin": "LAS", "dest": "RNO", "crs_dep_time": "2200",  
  "delay_probability": 0.9745, "risk_band": "0.9-1.0", "threshold": 0.5,  
  "issued_at": "2024-09-09T05:00:00+00:00", "mode": "replay"}
```

At that threshold, 54.9% of alerts are real delays against a 13.8% base rate: an alert is four times more likely to be a delay than a randomly chosen flight.

2. Data and event contract

Sources (full provenance, ownership, rights, and limitations in `docs/data_sources.md`): BTS On-Time Performance for flights, with one held-out week committed as the demo sample; NOAA ISD hourly surface observations for actual weather; IEM archived TAFs for forecast weather, used only by the substitution study in section 4. Limitations: outcomes arrive hours after departure, so truth is late by nature, and 0.34% of flights have no tail number.

The event is one departure. A real one:

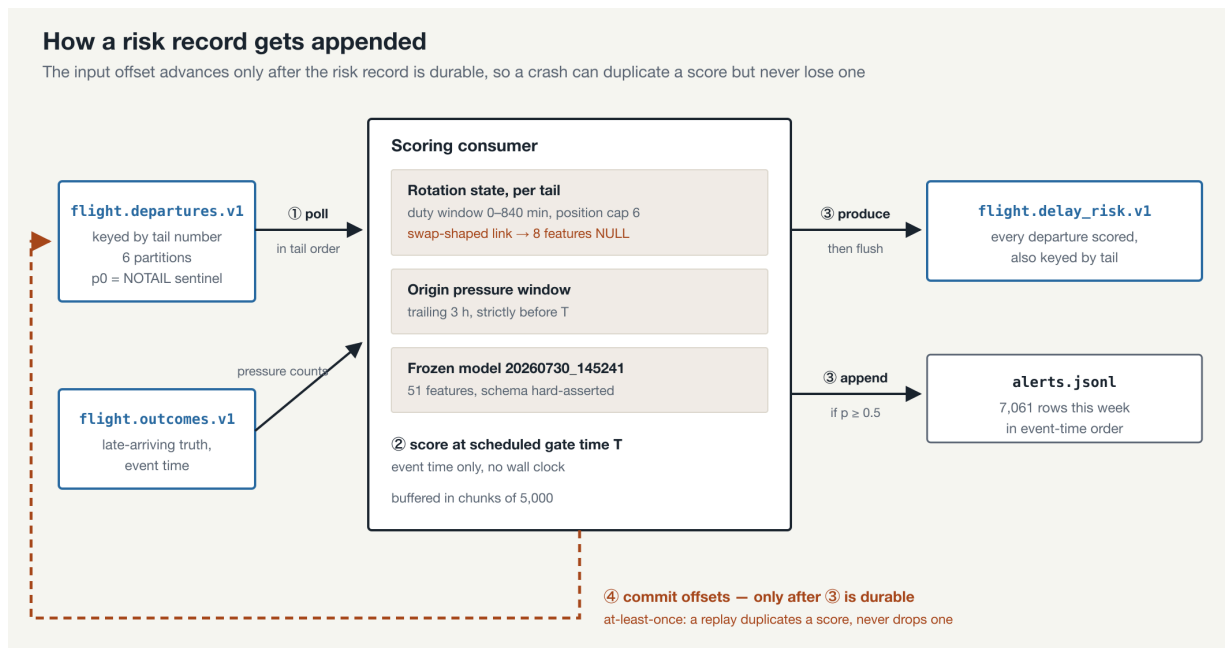
```
{"flight_date": "2024-09-02", "carrier": "WN", "flight_number": "469",  
  "origin": "OAK", "dest": "SAN", "crs_dep_time": "0610",
```

```
"crs_dep_ts_utc": "2024-09-02T13:10:00Z", "crs_arr_time": "0745",  
"crs_elapsed_min": 95, "distance_mi": 446, "tail_number": "N238WN",  
"mode": "replay", "is_warmup": false}
```

The contract: Avro schemas in the Confluent Schema Registry with BACKWARD compatibility, the only definition of the event; there is no application-side schema copy. If a record does not match, it never gets into Kafka. Every field is tagged with when it becomes knowable (`knowable_at` in {`schedule`, `pre_departure_stream`, `post_departure`}).

The key: tail number. Per-tail ordering is what makes rotation state possible; the same aircraft's flights land in the same partition, in order. Null tails route to a `NOTAIL` sentinel partition. Three topics share this key: `flight.departures.v1` (schedule fields only), `flight.delay_risk.v1` (the scores), and `flight.outcomes.v1` (late-arriving truth). Everything downstream depends on what goes into this event, and on what is left out.

3. Architecture and working implementation



Pipeline

One path: the replay producer streams the week into Kafka in departure order; the scoring consumer builds each flight's features, scores it with the frozen model, and publishes to the risk topic, appending high-risk flights to the alert file; hours later the outcomes producer emits the truth at actual-arrival event time, and the evaluator joins outcomes back to predictions. Three decisions make it work:

- Order. Within each tail's partition, which is all the ordering the rotation state tracking needs.
- Resume. Score and produce first, commit second. A crash replays, never loses: at-least-once, with duplicates counted by the evaluator.
- Reproducible. One command reproduces the whole pipeline, byte-identical every run. Local Docker Compose, no cloud, zero cost.

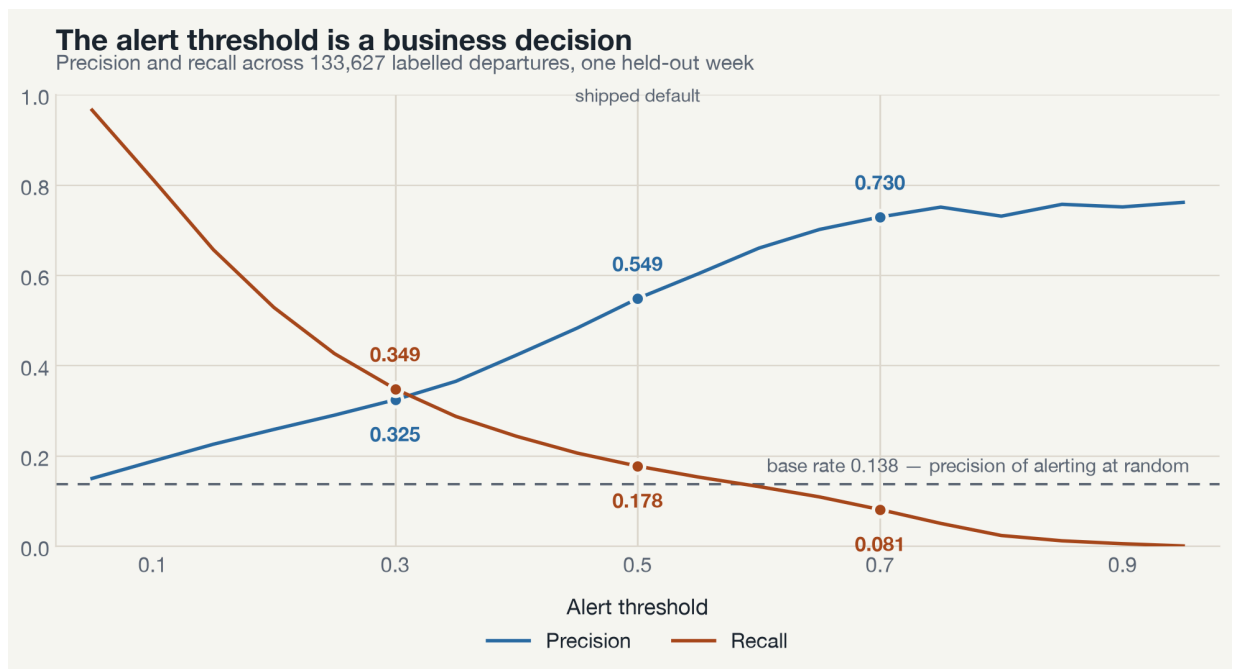
One flight through the system: a departure event arrives and the consumer reads its tail's rotation state (the inbound leg, if the linkage is schedule-consistent), assembles the 51-feature frame from pre-departure-knowable fields only, scores it with the frozen classifier plus Platt calibration, produces the risk record, appends the alert row if $p \geq 0.5$, and only then commits the input offset. That ordering is deliberate: a crash can duplicate a score, never lose one. State and time: per-tail rotation state inside the 840-minute duty window, event time only, no wall clock in any scored field.

The leakage rule is the design's core finding, and nothing post-departure reaches the scorer. Predictors may use only information knowable before departure; anything realized at or after departure is a label, never a feature. The rule extends to linkage, not just values: rotation features chain a flight to its inbound leg by the tail that actually operated it, and a day-of plane swap restructures that chain, so the link itself is a day-of outcome. Swap-shaped links get NULL rotation features, in training and in the stream. Enforcement is tested, not promised: a contract test checks `knowable_at` against the live registry, the leakage suite proves no forbidden field reaches the scorer, and a full-week rotation parity check proves the stream's chain matches training's.

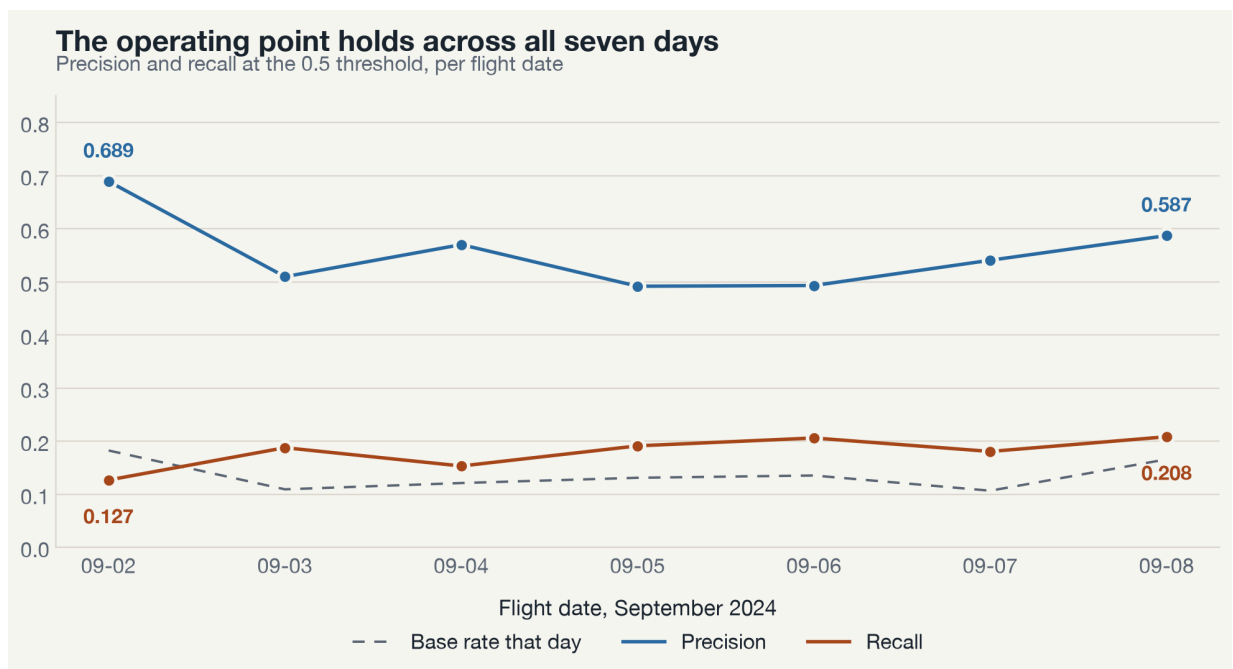
4. Evidence and reproducibility

The evaluator joins every scored flight to its real outcome and reports alert quality over the week (`data/reference_output/streaming_eval.json`). Every one of the 151,878 risk events lands in exactly one counter, a sum the conservation test enforces: 133,627 evaluated, 17,780 warm-up day, 248 cancelled, 222 diverted without a label, 1 orphan, 1 past the 48-hour TTL.

Metric	Value
Precision at $p \geq 0.5$	0.549 (5,950 evaluated alerts)
Recall at $p \geq 0.5$	0.178
Precision at $p \geq 0.7$	0.730
PR-AUC	0.340 (base rate 0.138)
Calibration error (10-bin ECE)	0.033

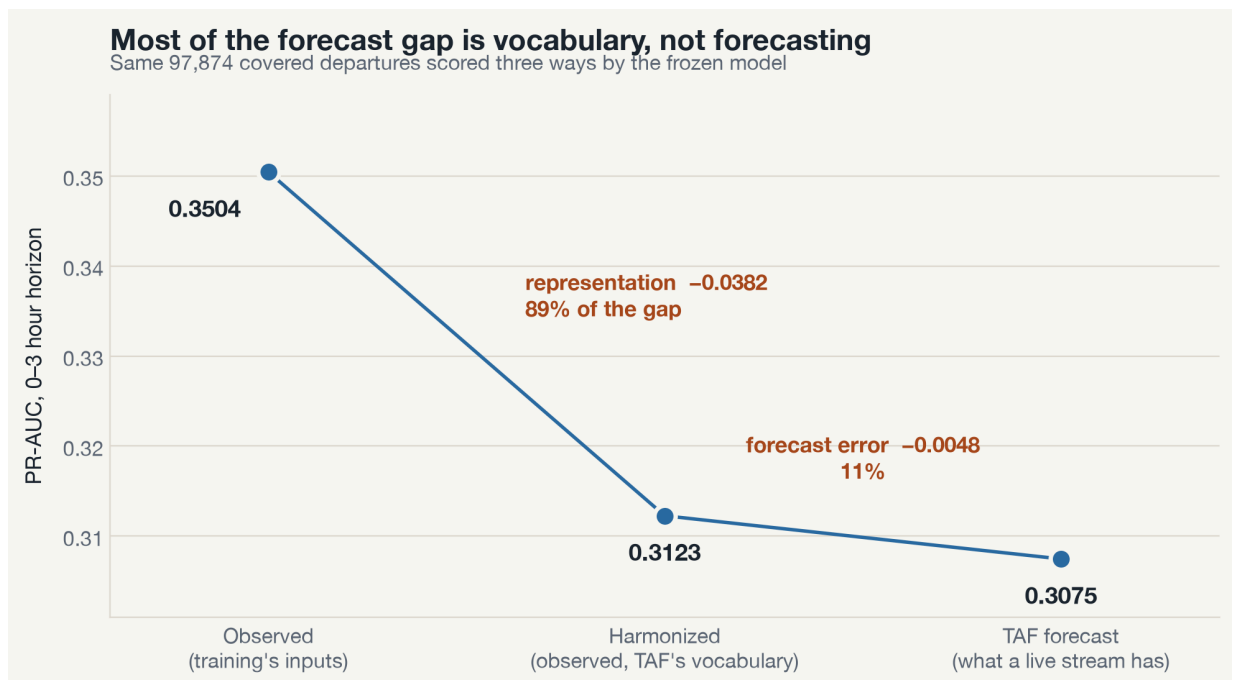


Threshold sweep



By day

The second finding: training saw observed weather, but a live stream only has forecasts. We measured that substitution instead of assuming it. The short-horizon cost is 0.043 PR-AUC, but degrading observed weather into the forecast's vocabulary shows 89% of that gap is representation mismatch and only 0.005 is genuine forecast error, below the pre-registered retrain trigger. The fix is harmonizing features at the source, not retraining.



Weather gap

The engineering lesson was harsher: the replay's event-time timestamps are from 2024, and Kafka's default retention deleted the whole replay week minutes after startup. Every test passed warm; only a cold-machine rehearsal caught it. The broader lesson: serve-time leakage is harder than train-time leakage, because linkage leaks even when values do not, and the stricter rule still kept 89% of the rotation signal. Next step: replace the replay producer with live flight and weather feeds.

AI role: disclosed and verified AI-assisted development, bounded by the handoff prompts in `docs/HANDOFF_PROMPTS.md` and gated by parity and byte-identity checks; task, evidence, decisions, verification, and limitations are in `AI_USAGE.md`. Individual contributions are in `docs/CONTRIBUTIONS.md`: the work split at the schema, Sebastian upstream (contracts, producers, evaluator, platform), Aidan downstream (consumer, rotation, alerts, TAF study).

Review path (README quickstart, 9 steps, no cloud credentials, zero spend): `uv sync`, fetch the model artifacts, `make demo` brings up the stack, registers contracts, replays the week, scores it, and prints the evaluation; expected output is stated verbatim in the README; `make test` runs the 59-test suite; `make down` cleans up. Because replay is deterministic, a diff against the committed reference output is the acceptance check.